



PROGRAMACIÓN

1º DAM

RETO 6



ENUNCIADO

Se quiere comprobar si hay **black SEO** en diferentes páginas web. Esta técnica maliciosa es utilizada para manipular los resultados de los motores de búsqueda de manera no ética. Se realiza insertando contenido oculto, enlaces fraudulentos o redirigiendo a usuarios y bots de búsqueda a diferentes versiones de una página web con el fin de engañar a los motores de búsqueda

Para ello vamos a hacer un programa en Java que me compruebe varios elementos. Para ello tenemos que tener cuidado con esto:

```
<div style="display:none;">Palabras clave ocultas</div>
<span style="font-size:0px;">Texto invisible</span>
<a href="spamlink.com" style="visibility:hidden;">Enlace SPAM</a>
```

RESOLUCIÓN

El **Black Hat SEO** hace referencia a un conjunto de técnicas de optimización web no éticas y engañosas para manipular los algoritmos de búsqueda y obtener un posicionamiento rápido, violando las directrices de los motores de búsqueda como Google (a diferencia del White Hat SEO que sigue las normas para un éxito a largo plazo).

Para resolver el ejercicio creamos un programa en java al que, pasándole el nombre de la página .html a analizar, sea capaz de leer el contenido y detectar patrones que correspondan a alguna de las técnicas de Black Hat SEO a las que hace referencia la imagen del enunciado:

- > Palabras clave ocultas mediante el atributo y valor **display:none.**
 - > Texto invisible haciendo nulo el tamaño de letra **font-size:0px.**
 - > Enlaces con visibilidad oculta mediante **visibility:hidden.**
- Estos patrones ocultan contenido a los usuarios pero no a los bots de búsqueda.

Puesto que los archivos .html son ficheros de texto plano (contienen exclusivamente caracteres de texto) podemos realizar el programa para llevar a cabo la lectura del archivo correspondiente mediante las clases **FileReader** y **BufferedReader**:

- Mediante la primera construimos un objeto que almacena el archivo (que pasamos como parámetro en forma de String) y
- Mediante la segunda construimos un objeto a partir del anterior, que en este caso llamamos **texto** y que contendrá todo el texto del archivo que le pasemos.

Seguidamente declararemos una variable de tipo String que llamamos **línea** en la que se irá almacenando cada línea del contenido del archivo para cada iteración de un **while**; mediante el while y usando como condición que la siguiente línea tenga contenido iteramos a lo largo de todo el texto:

- > Para extraer el contenido de la línea se emplea el método **.readLine()** de la clase **BufferedReader**.
- > En cada iteración y empleando condicionales if y el método **.contains()** de la Clase String (puesto que cada línea extraída es un objeto de este tipo), comparamos con los textos de mala práctica buscados (previamente pasados a minúsculas para que no haya problemas con la sensibilidad a las mayúsculas).

Informaremos por pantalla de si no se encuentra código malicioso o si, a la vez de la línea en que aparece (caso de haberlo); para esto nos valemos de una variable de conteo auxiliar **numLinea** y de un boolean denominado **encontrado** (true si se encuentra código malicioso).

Con el fin de manejar errores el código se implementa en un try-catch-resources (se mete entre paréntesis la creación de los objetos para que Java cierre los recursos automáticamente al terminar). El bloque tendrá dos catch; uno para capturar excepciones por archivos no encontrados **FileNotFoundException** y otro para la captura de excepciones por errores en la entrada/salida de datos **IOException**.



```

package reto6;

//Importaciones con las clases para lectura de archivos y manejo de excepciones
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.IOException;

public class Principal {

    public static void main(String[] args) {

        //Nombre del fichero de la página que queremos analizar
        String nombreFichero = "index_bueno.html";

        //try-with-resources
        //Metemos en el try la apertura de recursos (FileReader y BufferedReader)
        //Los metemos entre los () del try para que se Java los cierre automáticamente
        try (BufferedReader texto = new BufferedReader(new FileReader(nombreFichero))) {
            String linea;

            //Variable para indicar la línea del fichero en que se encuentra
            //si lo hay ,el código malintencionado
            int numLinea = 1;

            //Variable que recoge si se ha encontrado o no el código malicioso
            boolean encontrado = false;

            while ((linea = texto.readLine()) != null) { //Almacenamos en linea el texto leído en la línea
                                                        //y si no es nulo ejecutamos las órdenes del while
                //Pasamos a minúsculas todo para evitar problemas de sensibilidad a mayúsculas
                linea = linea.toLowerCase();

                //Comprobación de técnicas maliciosas
                //Comprobación de palabras clave ocultas
                if (linea.contains("display:none")) {
                    System.out.println("Código de Técnica maliciosa detectado (Black Hat SEO)");
                    System.out.println("Posibles palabras ocultas");
                    System.out.println("style-->display:none --> en la línea " + numLinea);
                    encontrado = true;
                }
                //Comprobación de texto invisible
                if (linea.contains("font-size:0px")) {
                    System.out.println("Código de Técnica maliciosa detectado (Black Hat SEO)");
                    System.out.println("Posible texto invisible");
                    System.out.println("style-->font-size:0px --> en la línea " + numLinea);
                    encontrado = true;
                }
                //Comprobación de enlace oculto
                if (linea.contains("visibility:hidden")) {
                    System.out.println("Código de Técnica maliciosa detectado (Black Hat SEO)");
                    System.out.println("Posible enlace (SPAM) oculto");
                    System.out.println("style-->visibility:hidden --> en la línea " + numLinea);
                    encontrado = true;
                }
                numLinea++;
            }

            if (!encontrado) {
                System.out.println("No se ha encontrado código de técnicas maliciosas (Black Hat SEO)");
            }

        } catch (FileNotFoundException exFNF) {
            //Excepción --> error al intentar abrir un fichero que no encuentra
            System.out.println(exFNF.getMessage());
        } catch (IOException exIOE) {
            //Excepción --> error en operaciones de entrada/salida
            System.out.println(exIOE.getMessage());
        }
    }
}

```



```
package reto6;

//Importaciones con las clases para lectura de archivos y manejo de excepciones
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.IOException;

public class Principal {

    public static void main(String[] args) {

        //Nombre del fichero de la página que queremos analizar
        String nombreFichero = "index_bueno.html";

        //try-with-resources
        //Metemos en el try la apertura de recursos (FileReader y BufferedReader)
        //Los metemos entre los () del try para que se Java los cierre automáticamente
        try (BufferedReader texto = new BufferedReader(new FileReader(nombreFichero))) {
            String linea;

            //Variable para indicar la línea del fichero en que se encuentra
            //si lo hay ,el código malintencionado
            int numLinea = 1;

            //Variable que recoge si se ha encontrado o no el código malicioso
            boolean encontrado = false;

            while ((linea = texto.readLine()) != null) { //Almacenamos en linea el texto leído en la línea
                //y si no es nulo ejecutamos las órdenes del while

                //Pasamos a minúsculas todo para evitar problemas de sensibilidad a mayúsculas
                linea = linea.toLowerCase();

                //Comprobación de técnicas maliciosas
                //Comprobación de palabras clave ocultas
                if (linea.contains("display:none")) {
                    System.out.println("Código de Técnica maliciosa detectado (Black Hat SEO)");
                    System.out.println("Posibles palabras ocultas");
                    System.out.println("style-->display:none --> en la línea " + numLinea);
                    System.out.println("");
                    encontrado = true;
                }
                //Comprobación de texto invisible
                if (linea.contains("font-size:0px")) {
                    System.out.println("Código de Técnica maliciosa detectado (Black Hat SEO)");
                    System.out.println("Posible texto invisible");
                    System.out.println("style-->font-size:0px --> en la línea " + numLinea);
                    System.out.println("");
                    encontrado = true;
                }
            }
        }
    }
}
```





```
//Comprobación de enlace oculto
if (linea.contains("visibility:hidden")) {
    System.out.println("Código de Técnica maliciosa detectado (Black Hat SEO)");
    System.out.println("Posible enlace (SPAM) oculto");
    System.out.println("style-->visibility:hidden --> en la línea " + numLinea);
    System.out.println("");
    encontrado = true;
}
numLinea++;
}

if (!encontrado) {
    System.out.println("No se ha encontrado código de técnicas maliciosas (Black Hat SEO)");
}

} catch (FileNotFoundException exFNF) {
    //Excepción --> error al intentar abrir un fichero que no encuentra
    System.out.println(exFNF.getMessage());
} catch (IOException exIOE) {
    //Excepción --> error en operaciones de entrada/salida
    System.out.println(exIOE.getMessage());
}
}
}
```



ARCHIVOS HTML PARA COMPROBACIÓN

> Escribimos dos archivos .html sencillos, uno con buenas prácticas y otro con prácticas black hat para comprobar que nuestro programa funcione. El código de los archivos sería el siguiente:

Index_malo.html

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Archivo con Black Hat SEO</title>
</head>
<body>
<h1>MÓVILES AL MEJOR PRECIO</h1>
<p>Compra barato</p>
<div style="display:none;">comprar venta gratis dinero ganar rapido</div>
<p>Últimas tecnologías <span style="font-size:0px;">móviles robados sin rastreo</span></p>
<p>Contacto: <a href="http://spamlink.com" style="visibility:hidden;">Este enlace no se ve pero te estafa</a></p>
</body>
</html>
```

Index_bueno.html

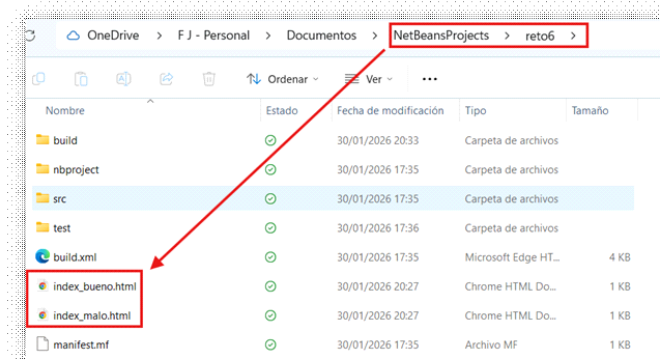
```
<!DOCTYPE html>
<html lang="es">

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Archivo sin Black Hat SEO</title>
</head>

<body>
<h1>BLOG RUTAS DE CICLOTURISMO EN TOLEDO</h1>
<p>Las mejores rutas para aficionados</p>
<div style="color: blue; font-size: 14px;">
  ¡¡¡Contenido gratuito!!!
</div>
<br>
<a href="https://ciclismotoledo.es">Visita nuestras rutas</a>
</body>

</html>
```

> Para su correcta lectura los archivos los colocamos en la misma carpeta del proyecto de NetBEans:





> Ejecutamos el programa:

> La primera vez pasándole como archivo "index_malo.html":

```
public class Principal {  
  
    public static void main(String[] args) {  
  
        //Nombre del fichero de la página que queremos analizar  
        String nombreFichero = "index_malo.html";  
    }  
}
```

> Resultado de la ejecución:

```
Output - reto6 (run) X  
run:  
Código de Técnica maliciosa detectado (Black Hat SEO)  
Posibles palabras ocultas  
style-->display:none --> en la línea 11  
  
Código de Técnica maliciosa detectado (Black Hat SEO)  
Posible texto invisible  
style-->font-size:0px --> en la línea 12  
  
Código de Técnica maliciosa detectado (Black Hat SEO)  
Posible enlace (SPAM) oculto  
style-->visibility:hidden --> en la línea 13  
  
BUILD SUCCESSFUL (total time: 0 seconds)
```

> Observamos que detecta los códigos buscados e indica las líneas exactas en que se hallan.

> Ahora repetimos pasándole como archivo "index_bueno.html":

```
public class Principal {  
  
    public static void main(String[] args) {  
  
        //Nombre del fichero de la página que queremos analizar  
        String nombreFichero = "index_bueno.html";  
    }  
}
```

> Resultado de la ejecución:

```
Output - reto6 (run) X  
run:  
No se ha encontrado código de técnicas maliciosas (Black Hat SEO)  
BUILD SUCCESSFUL (total time: 0 seconds)
```

> Observamos que en este caso encuentra que el código no tiene las prácticas maliciosas buscadas.

> Posibles mejoras:

> Hay varias maneras de escribir la combinación de atributo:valor para **font-size:0px** se podrían emplear expresiones llamadas Expresiones Regulares (Regex) para tener en cuenta los posibles espacios en blanco entre : y 0, la posibilidad de usar px o no. Para ello habría que importar las correspondientes clases específicas.